

Bitácora de uso de IA

Trabajamos con Claude Code (Anthropic) como asistente durante todo el proyecto: diseño, código, pruebas y documentación. Esto es lo que le pedimos, lo que propuso, lo que aceptamos y lo que cambiamos.

1. Primero la arquitectura, antes de tener el enunciado. Le pedimos describir la arquitectura y las restricciones de una API que recibe una especie y tres variables ambientales, con SOLID y MVC, y que plantara a propósito un error de arquitectura sin decir cuál, para buscarlo nosotros. Propuso capas con puertos y adaptadores, un dominio sin framework y los estados de la planta como estrategias. **Aceptamos la estructura tal cual.** El error lo encontramos con una pista: un `ServicioDiagnostico` que dependía del repositorio de Spring Data en lugar del puerto del dominio (inversión de dependencias rota). La IA nos mostró que la misma línea rompía una segunda regla: la entidad JPA salía de infraestructura.

2. Una prueba de arquitectura que mentía. Construyó el prototipo con reglas ArchUnit. Al aplicar la versión con el error, la regla que debía cazarlo **pasó en verde**: prohibía el paquete `org.springframework.data`, y el código violador nombraba una clase nuestra que vive en infraestructura. Se agregó la regla `la_aplicacion_no_depends_de_infraestructura`. Aprendimos que una prueba de arquitectura vale por lo que comprueba, no por lo que su nombre promete.

3. Llegó el enunciado y el diseño no alcanzaba. Le pasamos el enunciado y le pedimos comparar. Nos mostró dos problemas de fondo: (a) nuestro diseño devolvía **un** estado por prioridad y descartaba el resto, pero RF2 y RF3 piden clasificar **cada** parámetro y agregar; (b) los tres parámetros eran campos de una clase, así que agregar pH habría tocado media aplicación. **Cambiamos:** los parámetros pasaron a ser datos (`Magnitud`, `Lectura`, `Medicion`) y la cadena de estados se reemplazó por clasificación + regla de agregación.

4. La regla de agregación la decidimos nosotros. La IA propuso tres reglas: por conteo, por desviación y "la temperatura manda". **Elegimos desviación** (CRITICO si un parámetro se sale más del 25 % del ancho de su rango), porque mide qué tan lejos está cada parámetro. Descartamos "la temperatura manda" porque nombra una magnitud concreta y un parámetro nuevo obligaría a abrir la regla.

5. Lo que corregimos de la IA.

- Un agente afirmó que agregar pH tocaría 2 archivos. Al revisar vimos que el DTO y el controlador todavía nombran las tres magnitudes: la cuenta real es mayor. El dominio no se toca; el borde web sí. Quedó documentado como tensión.
- Nos recomendó un solo repositorio con dos carpetas. **Decidimos dos repositorios**, front y back, por el criterio del curso y para que la frontera del cliente independiente sea visible.
- Su primera versión exponía el diagnóstico por `GET` con parámetros en la URL. **Pasamos a `POST` con cuerpo JSON** para seguir el contrato sugerido.
- Al probar el front contra un servidor simulado aparecieron dos bugs en código generado: los mensajes de error salían vacíos, y un `display: flex` dejaba visibles los banners de error aunque estuvieran ocultos. Se corrigieron antes del commit.

6. Sobre el historial. El prototipo del punto 1 quedó al principio en un único commit. Lo partimos en las etapas reales en que se construyó (andamiaje, dominio, pruebas, infraestructura, web, corrección de columnas, regla ArchUnit), con su fecha real (8-sep). No inventamos fechas. Todo el trabajo desde el enunciado se commiteó a medida que se hizo.

Criterio que usamos para aceptar o cambiar: si una propuesta obligaba a que el dominio supiera de HTTP, del CSV o de la base de datos, o a modificar una clase existente para agregar un parámetro o una regla, la cambiamos.